

Assignment #2



- 과목명: 자료구조
- 담당교수님: 김무철 교수님
- 학과: 소프트웨어학부
- 보고서작성자: 서정민

1. Assignment 2-1.c – BST 구현

```
10 Node* Insert(Node* node, int data)
11 {
12     if (node == NULL)
13     {
14         Node* newNode = (Node*)malloc(sizeof(Node));
15         newNode->data = data;
16         newNode->Left = NULL;
17         newNode->Right = NULL;
18     }
19     return newNode;
20 }
21 else
22 {
23     // edit1
24     /*
25     TODO: Fill the conditions in Insert function
26     */
27     // 삽입할 데이터가 현재 노드의 데이터보다 작다면
28     // 왼쪽으로 가서 다시 삽입 연산
29     if (data < node->data)
30     {
31         node->Left = Insert(node->Left, data);
32         return node;
33     }
34     /*
35     TODO: Fill the conditions in Insert function
36     */
37     // 삽입할 데이터가 현재 노드의 데이터보다 크다면
38     // 우측으로 가서 다시 삽입연산
39     else if (data > node->data)
40     {
41         node->Right = Insert(node->Right, data);
42         return node;
43     }
44     else
45     {
46         return node;
47     }
48 }
49 }
50 }
```

Insert 함수

삽입할 데이터가 현재 노드의 데이터보다 작다면, 왼쪽으로 가서 다시 Insert 함수를 호출하고, 삽입할 데이터가 현재 노드의 데이터보다 크다면 우측으로 가서 다시 Insert 함수를 호출한다.

```

53 int Search(Node* node, int data)
54 {
55     if (node == NULL) {
56         printf("No Node %d ", data);
57         return 0;
58     }
59
60     if (node->data == data)
61     {
62         return node->data;
63     }
64     else
65     {
66         /*
67         TODO: Implement the Search function conditions
68         */
69         // 찾는 데이터보다 지금 노드의 데이터가 작거나 같다면
70         // 왼쪽으로 가야 한다
71         if (node->data > data) {
72             return Search(node->Left, data);
73         }
74         else {
75             return Search(node->Right, data);
76         }
77     }
78 }
79

```

Search 함수

Else 부분에서 찾는 데이터보다 지금 노드의 데이터가 작거나 같다면, 왼쪽 노드로 가야 하고, 그 외의 경우는 오른쪽 노드로 가서 찾아야 한다.

```

81 Node* Delete(Node* node, int data)
82 {
83     if (node == NULL) return NULL;
84
85     if (node->data == data)
86     {
87         Node* deleteNode = node;
88         /*
89         TODO: Implement the Delete function conditions
90         */
91         // 1. 자식이 없는 노드의 경우 - 그냥 지우자
92         if (node->Left == NULL && node->Right == NULL) {
93             free(deleteNode);
94             return NULL;
95         }
96         // 2. 자식이 하나만 있는 노드의 경우, 크기 판단해서 그 위에 붙이기
97         else if (node->Left == NULL || node->Right == NULL) {
98             Node* tmp = NULL;
99             if (node->Left != NULL) {
100                 // 왼쪽노드가 있을 경우
101                 tmp = node->Left;
102             }
103             else {
104                 //오른쪽 노드가 있을 경우
105                 tmp = node->Right;
106             }
107             free(deleteNode);
108             return tmp; //그 자식노드를 대신 붙인다
109         }

```

Delete 함수

자식이 없는 노드라면 그냥 지운다, 자식이 하나만 있는 노드의 경우 그 자식노드를 위에 붙이고 지운다.

```
110 // 3. 3. 자식이 두개 다 있는 노드를 지울때 - 오른쪽 자식중 가장 작은걸 왼쪽 자식으로
111 else {
112     // 오른쪽 자식중 가장 작은 값 찾기
113     Node* parent = node;
114     Node* right_min = node->Right;
115     while (right_min->Left != NULL) {
116         parent = right_min;
117         right_min = right_min->Left;
118     }
119     node->data = right_min->data;
120
121     if (parent != node) {
122         parent->Left = right_min->Right;
123     }
124     else {
125         parent->Right = right_min->Right;
126     }
127     free(right_min);
128 }
129 return node;
130 }
```

자식이 두개 이상 있는 노드의 경우, 오른쪽 자식중 가장 작은 노드를 찾는다. 현재 노드의 데이터를 오른쪽 자식 중 가장 작은 노드의 데이터로 바꾸고, 가장 작은 노드의 부모와 연결한다

```
131 else
132 {
133     /*
134     TODO: Implement the delete function traversal
135     일단 노드까지 찾아가는게 먼저 그 다음에 경우를 판단하자
136     */
137     if (node->data > data) {
138         node->Left = Delete(node->Left, data);
139     }
140     else if (node->data < data) {
141         node->Right = Delete(node->Right, data);
142     }
143
144     return node;
145 }
146 }
```

현재 노드가 삭제할 노드가 아닌 경우, 찾아가면서 Delete 한다.

```

    20
  10   32
 4 13 25 55

No Node 12 Node: 0
Node 13
    20
  10   55
 4 13 25
|

```

실행 결과

2. Assignment 2-2.c – TRIE 트리 구현

```

26 // Function to insert a word into the trie
27 void Insert(TrieNode* root, const char* key) {
28     TrieNode* crawler = root;
29     while (*key) {
30         /*
31          * TODO: write a traversal on the key for inserting nodes in tree
32          */
33         int idx = *key - 'a';
34         if (crawler->children[idx] == NULL) {
35             crawler->children[idx] = createNode();
36         }
37         crawler = crawler->children[idx];
38         key++;
39     }
40     crawler->isEndOfWord = true;
41 }

```

Insert 함수

현재 문자를 알파벳 배열의 인덱스로 바꾸고, 현재 문자 인덱스에 해당하는 자식노드가 없으면 새로 생성한다. Crawler를 자식 노드로 갱신하여 다음 문자를 처리하고, key++ 연산을 하여 다음 문자로 이동한다. 문자열 맨 끝에 도달했다면 반복문에서 나오고, isEndOfWord 값을 true로 바꾼다.

```

42 // Function to search a word in the trie
43 bool search(TrieNode* root, const char* key) {
44     TrieNode* crawler = root;
45     while (*key) {
46         /*
47          * TODO: write a traversal on the key for searching nodes in tree
48          */
49         int idx = *key - 'a';
50         if (crawler->children[idx] == NULL) {
51             return false;
52         }
53         crawler = crawler->children[idx];
54         key++;
55     }
56     return (crawler != NULL && crawler->isEndOfWord);
57 }

```

Search 함수

root 노드부터 찾아나간다. 만약 현재 문자의 인덱스에 해당하는 자식노드가 없는 상태면 false 반환, 아닐경우 다음 문자로 이동하면서, 자식 노드로 이동하면서 찾는다. 문자열의 끝에 도달시 현재 노드가 NULL 이 아니고, isEndOfWord 가 true 라면 true 를 반환한다. 아닐경우 false 반환.

```

66 bool deleteHelper(TrieNode* root, const char* key, int depth) {
67     if (!root) {
68         return false;
69     }
70
71     // If last character of key is being processed
72     if (*key == '\0') {
73         if (root->isEndOfWord) {
74             root->isEndOfWord = false;
75             // If root does not have any children (its not prefix of any other word)
76             if (isEmpty(root)) {
77                 return true; // Delete the node
78             }
79             return false; // Do not delete the node
80         }
81     }
82     else {
83         int index = *key - 'a';
84         if (root->children[index]) {
85             /*
86              * TODO: write a traversal on the key for searching nodes and deleting
87              * Hint: recursive solutions are always in handy
88              */
89             bool delChild = deleteHelper(root->children[index], key + 1, depth + 1);
90             if (delChild) {
91                 free(root->children[index]);
92                 root->children[index] = NULL;
93
94                 if ((isEmpty(root)) && !(root->isEndOfWord))
95                     return true;
96             }
97         }
98     }
99     return false;
100 }
101

```

DeleteHelper 함수

루트 노드가 Null 일 경우, false 반환한다.

문자열에 끝에 도달했을 경우, 현재 노드가 단어의 끝이라면 isEndOfWord 를 false 로 만들고, 현재 노드에 자식노드가 없다면 true 를 반환해서 노드를 삭제한다. 아니라면 false 를 반환해서 노드를 삭제하지 않는다.

문자열에 끝에 도달하지 않은 경우, 현재 문자를 인덱스로 바꾸고, 현재 문자에 해당하는 자식 노드가 있는 경우, 재귀적으로 deleteHelper 함수 호출하여 다음 문자를 처리함. 자식 노드를 삭제할수 있는 경우, 자식노드를 삭제하고 그 포인터를 NULL 로 바꾼다. 현재 노드가 자식노드가 없고, 단어의 끝이 아니라면 true 를 반환해서 삭제하도록 한다.

```
103 void Delete(TrieNode* root, const char* key) {
104     if (root != NULL && *key && search(root, key)) {
105         deleteHelper(root, key, 0);
106     }
107     else {
108         printf("Cannot delete '%s': Word not found in Trie.\n", key);
109     }
110 }
```

Delete 함수

deleteHelper 함수, Search 함수를 이용해서 단어가 존재한다면 삭제한다.

```
116 void suggestions(TrieNode* root, const char* currentPrefix) {
117     if (root->isEndOfWord) {
118         printf("%s\n", currentPrefix);
119     }
120
121     if (isEmpty(root)) {
122         return;
123     }
124
125     for (int i = 0; i < ALPHABET_SIZE; i++) {
126         if (root->children[i]) {
127             /*
128              * TODO: write a traversal on the prefix for searching nodes and completing
129              * Hint: recursive solutions are always in handy
130              */
131             char newPrefix[100];
132             strcpy_s(newPrefix, sizeof(newPrefix), currentPrefix);
133             int length = strlen(currentPrefix);
134             newPrefix[length] = i + 'a';
135             newPrefix[length + 1] = '\0';
136
137             suggestions(root->children[i], newPrefix);
138         }
139     }
140 }
141 }
```

Suggestions 함수

주어진 접두사로 시작하는 단어들을 Trie 에서 찾아서 출력한다. 현재 노드가 단어의 끝이라면 currentPrefix 를 출력한다. 현재 노드가 비어있는 경우에는 그냥 return 한다.

현재 노드의 자식 노드가 존재하는 경우, 새로운 접두사를 현재 접두사 + 현재 알파벳에 종료 문자를 더해서 생성하고, 자식 노드에 대해 재귀적으로 Suggestions 함수를 호출한다.

```
143 void autoComplete(TrieNode* root, const char* prefix) {
144     TrieNode* crawler = root;
145     for (int i = 0; prefix[i] && crawler; i++) {
146         int index = prefix[i] - 'a';
147         crawler = crawler->children[index];
148     }
149
150     if (crawler) {
151         suggestions(crawler, prefix);
152     }
153 }
```

autoComplete 함수

접두사까지 탐색하고, 접두사에 해당하는 노드가 존재하면 suggestion 함수를 이용해서 단어를 출력한다.

```
Inserted: apple
Inserted: app
Inserted: application
Inserted: apricot
Inserted: banana
Inserted: berry
Inserted: blueberry
Inserted: blackberry
Inserted: banana
Inserted: balm
Inserted: ba
Autocomplete results for 'app':
app
apple
application
Autocomplete results for 'ba':
ba
balm
banana
Modified 'banana' to 'bandana'.
Autocomplete results for 'ba':
ba
balm
bandana
Cannot delete 'banana': Word not found in Trie.
Cannot delete 'banana': Word not found in Trie.
```

실행 결과